Major Project Progress Report on

Analytics, Reporting & Management Monitoring System with AI/ML Insights

By

Aswin Chettri (202422006)

In partial fulfilment of requirements for the award of degree in

Masters of Computer Application (MCA)

(2024-2026)

Under the Project Guidance of

Dr. Moumita Pramanik, Assistant Professor - I, Dept. of CA

Mr. Somnath Barik, Senior Developer, Euphoria GenX (External Guide)

Department of Computer Applications
Sikkim Manipal institute of Technology
Majhitar, Rangpo, East Sikkim

DEPARTMENT OF COMPUTER APPLICATIONS
SIKKIM MANIPAL INSTITUTE OF TECHNOLOGY
(A constituent college of Sikkim Manipal University)
MAJITAR, RANGPO, EAST SIKKIM – 737135

***Abstract*:** Gov-Vision is an enterprise-grade web platform that digitizes how organizations create, review, approve, track and audit important decisions. It replaces manual emails and paper-based approvals with a structured workflow, ensuring transparency, governance, and real-time analytics. This module focuses on the Analytics, Reporting & Management Monitoring System with AI/ML Insights. It acts as the analytics and monitoring layer, providing management and leadership with real-time dashboards, AI-driven anomaly detection, decision performance metrics, risk distribution insights, and compliance monitoring. By combining classical business intelligence with three targeted AI/ML capabilities: anomaly detection, predictive delay forecasting, and AI-driven risk scoring, the module enables organizations to move from responding to problems to preventing them. The system supports trend-based analysis, automated custom reporting, and exportable summaries for informed managerial decision making. By converting decision records into charts and summaries and reports, the module enables data-driven governance, performance evaluation, and strategic oversight across departments.

LIST OF CONTENTS

Chapter	Title	Page No
	ABSTRACT	
1	**INTRODUCTION**	**1-15**
1.1	General overview of the problem	2
1.2	Feasibility Study	2
1.2	Literature Survey	3-4
1.3	Problem Definition	4
1.4	Analysis of the Problem	4-6
1.5	Solution Strategy	6-9
1.6	Software Requirement Specifications	9-15
1.6.1	Functional Requirements	9-14
1.6.2	Non-Functional Requirements	14-15
2	**PROJECT PLANNING**	**15-20**
2.1	Hardware and Software Requirements	15
2.2	Team Structure	16
2.3	SLDC	16
2.4	Gantt–chart	17
3	**DESIGN STRATEGY FOR THE SOLUTION**	**18**
3.1	Architecture Diagram	18
4	**PROGRESS TILL DATE**	**19-31**
4.1	Analytics Dashboard	19-20
4.2	Data Ingestion (CSV Pipeline)	20-22
4.3	. Database Model - MongoDB Collections	22-26
4.4	. API Testing- Thunder Client	26-27
4.5	. Isolation Forest Model Training	28-31
5	**SUMMARY AND CONCLUSION**	**32-34**
5.1	Progress Summary	32-33
5.2	Conclusion	33-34

LIST OF FIGURES

FIG. NO.	FIGURE NAME	PAGE NO.
1	Team Structure	16
2	Agile Software Development Life Cycle	16
3	Gantt Chart	17
4	Architecture Overview Diagram of Analytics, Reporting & AI/ML Monitoring System	18
5	Analytics Dashboard- KPI cards, chart panels, and real-time anomaly feed	19
6	Analytics Dashboard- Compliance Rate Trend, Decision Throughput, Avg Approval Time Distribution and anomaly feed	20
7	MongoDB – m1_decisions	23
8	MongoDB- m3_kpi_snapshots organization - level snapshot document	24
9	MongoDB- m3_anomalies document	25
10	Thunder Client- POST /ml/anomaly/predict with normal input values	27
11	Results from the Isolation Forest Model Training	29- 30
12	Feature Separation (Predicted Anomalies vs Normals)	31
13	Sanity Test Cases	31
14	Scatter Map (Cycle Time vs Rejection Count)	32

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
1	Literature Study	3-4
2	APIs of the KPI Dashboard Sub Module	10
3	APIs of the Anomaly Detection Sub Module	11
4	APIs of the Predictive Forecasting Sub Module	12
5	APIs of the Risk Scoring Sub-Module	13
6	APIs of the Report Generation Sub-Module	14
7	CSV Source Column to m1_decisions Schema Field Mapping	21
8	Department Normalization Mapping	21
9	m1_decisions - Field Definitions	22-23
10	m3_kpi_snapshots - Field Definitions	23-24
11	m3_anomalies - Field Definitions	25
12	Thunder Client API Test Results	26-27
13	Isolation Forest Severity Band Distribution	28
14	Gov-Vision Analytics Module - Progress Summary	14

1. INTRODUCTION

The Analytics, Reporting & Management Monitoring System with AI Insights is a dedicated module within the Gov-Vision Digital Decision & Governance Platform, an enterprise-grade system designed to bring analytical depth, transparency, and accountability to organizational decision making. As governance processes grow increasingly complex, this module exists to ensure that the right people have the right information, at the right time, supported by clearly defined Key Performance Indicators (KPIs) that measure governance effectiveness across the organization.

In modern enterprise environments, decision-making generates vast amounts of data that, when properly analysed, can reveal critical insights about organizational efficiency, risk exposure, and compliance status. Most organizations still rely on manual spreadsheets, delayed reports, and intuition-led decision, leaving bottlenecks undetected, compliance risks overlooked, and performance gaps unaddressed. This module directly solves that problem by serving as the analytical and visibility layer of the Gov-Vision platform, transforming raw decision data into clear insights for executives, managers, compliance officers, analysts, and department heads.

While the broader Gov-Vision platform manages the full lifecycle of organizational decisions from creation and approval through to compliance and auditing, this module focuses exclusively on analytics, reporting, and monitoring. It continuously tracks KPIs across decision workflows, departmental performance, and policy compliance, providing organizations with real time visibility into governance activity. It operates as an independent microservice, consuming processed decision data via secure internal REST APIs from the Decision Management and Governance & Compliance modules, and presenting that data through analytical dashboards and structured reports. It does not manage decisions; it analyses and monitors them.

The module's most notable capabilities include real-time executive dashboards that display decision metrics, approval timelines, KPI performance, and risk indicators, alongside AI-powered anomaly detection that automatically flags abnormal patterns in approval workflows. It features predictive analytics that forecast approval delays before they occur, and AI-driven risk scoring that assigns dynamic risk levels to departments and decision types. Users also benefit from comprehensive analytical tools for examining decision patterns, compliance status, and departmental performance, as well as customizable automated report generation with scheduled delivery in PDF, Excel, and CSV formats. Role-based access controls ensure that every stakeholder sees precisely the data relevant to their role.

1.1 General overview of the problem

Enterprise governance environments today lack a centralized, real-time analytical system for transforming decision-making data into usable management information. When organizations need visibility into approval workflows, KPI performance, or compliance status, the process relies on manually compiled spreadsheets and periodic reports assembled across disconnected systems. These methods are neither timely nor reliable. Governance data is operationally critical and without automated monitoring, bottlenecks go undetected, compliance risks are overlooked, and performance gaps remain unaddressed until they escalate.

Additionally, there is no mechanism through which recurring anomalies approval delay spikes, unusual rejection patterns, or sudden compliance deterioration are automatically flagged against historical departmental baselines. There is also no system to forecast potential workflow congestion before it occurs, nor a continuous risk scoring mechanism to classify departmental risk levels in real time. Without these capabilities, management is forced into a reactive posture, responding to failures only after they have materialized.

Furthermore, decision data is fragmented across multiple systems with no unified governance view available to executives or department heads. All these gaps collectively highlight the need for an integrated, AI/ML enhanced analytics and monitoring platform such as the Gov-Vision Analytics Module.

1.2 Feasibility Study

1.2.1 Technical Feasibility:

The module is technically feasible as it is built on modern web technologies well suited for scalable, data-driven, and AI integrated applications. The React.js frontend with Chart.js and Recharts handles complex real time visualizations. The Node.js/Express.js backend handles API orchestration. Python-based ML services (scikit-learn, Prophet) are integrated via REST for anomaly detection and forecasting. These are all well-documented, and widely adopted in enterprise environments

1.2.2 Operational Feasibility:

The system is designed for ease of use. Dashboards are role based and auto populate based on user designation. AI insights are surfaced as simple alerts and summaries not raw model outputs so no data expertise is required from users. This minimizes training requirements and simplifies system usage.

1.2.2 Economic Feasibility:

Development costs are manageable, primarily involving open-source technologies. The ML/AI components rely on open-source libraries (scikit-learn, Prophet, Transformers). Estimated development timeline is within the proposed project schedule.

1.3 Literature Survey

Author(s) & Year	Full Title	Key Findings	Relevance to Our Project	Gaps Identified
[1] Dheerendra Yaganti, 2020	A Modular Python-Based Framework for Real-Time Enterprise KPI Visualization Using Pandas and Interactive Dashboards	Proposed a modular analytics framework using Pandas for data processing, Matplotlib and Plotly for visualization, and API-based ingestion. Used structured KPI aggregation and dashboard rendering techniques.	Supports Analytics & Dashboard module, which uses structured data processing and visualization techniques. Aligns with modular architecture and real-time KPI analytics implementation.	No machine learning models used. No governance rule validation, predictive forecasting, risk scoring, or decision lifecycle workflow integration.
[2] Dachepalli, V. (2025)	AI-Driven Decision Support Systems in ERP	Used supervised machine learning models (regression, decision trees, ensemble methods) and time-series forecasting for predictive analytics within ERP systems.	Aligns with our AI Insights module, which also uses ML-based predictive analytics to improve enterprise decision quality.	No governance workflow, no policy validation engine, no audit trail, and no structured decision lifecycle integration.
[3] Herreros-Martínez A, Magdalena-Benedicto R, Vila-Francés J, et al., (2025)	Applied Machine Learning to Anomaly Detection in Enterprise Purchase Processes, A Hybrid Approach Using Clustering and Isolation Forest	Used Clustering (unsupervised learning) and Isolation Forest for anomaly detection in enterprise purchase data. Improved identification of irregular and fraudulent patterns.	Supports our AI Anomaly Detection module, which also applies Isolation Forest for governance anomaly flagging.	Limited to anomaly detection; no governance workflow, dashboards, forecasting, or decision lifecycle integration.
[4] Boning Huang et al., (2021)	Enterprise Risk Assessment Based on Machine Learning	Developed enterprise risk models using Random Forest, Support Vector Machine (SVM), and AdaBoost. Used structured financial indicators as features for supervised classification. Demonstrated improved risk prediction accuracy over traditional statistical models.	Strongly aligns with our AI Risk Scoring module, which uses Random Forest for governance risk classification. Similar supervised ML training on structured indicators.	Focused only on financial risk classification. No governance policy enforcement, workflow automation, audit tracking, or real-time enterprise dashboard integration.

[5] Andrea Kolková & Aleksandr Ključnikov, (2022)	Demand Forecasting: AI-Based, Statistical and Hybrid Models vs Practice-Based Models	Compared ARIMA, ETS, TBATS, Artificial Neural Networks (ANN), Hybrid models, and Prophet forecasting. Used RMSE, MAE, and MASE for evaluation. Found Prophet effective for enterprise-scale forecasting.	Justifies our use of Prophet model for Predictive Delay Forecasting. Supports time-series modeling and performance validation techniques in enterprise environments.	Focused only on demand forecasting. No governance engine, no compliance automation, no risk scoring integration, and no decision lifecycle management.

Table 1: Literature Study

1.4 Problem Definition

Organizational decision-making in enterprise governance environments generates structured, measurable data at every stage of execution. Each decision follows a defined multi-stage workflow, Example: *Submitted → Under Review → L1 Approval → L2 Approval → Approved or Rejected* producing quantifiable attributes such as timestamps, stage durations, reviewer identifiers, revision counts, rejection reasons, SLA deviations, and compliance indicators.

The core challenge is the absence of a system that continuously ingests, processes, and transforms this data into real-time managerial insights. While the Decision Management & Workflow System and the Governance, Policy & Compliance Engine digitize and store the complete decision lifecycle, the generated data remains underutilized. It is stored in MongoDB collections without an automated analytics pipeline to compute KPIs, detect anomalies, or predict future outcomes.

As a result, management relies on manually prepared reports and spreadsheets that are delayed, error-prone, and incapable of reflecting real-time operational status. Furthermore, the lack of applied machine learning techniques such as anomaly detection, predictive delay forecasting, and risk classification creates a critical gap in proactive governance.

The Analytics, Reporting & AI/ML Monitoring System is designed to address this gap by converting raw decision data into actionable, real-time intelligence.

1.5 Analysis of the Problem

A systematic analysis of the governance data pipeline within the Gov-Vision platform reveals six discrete failure points, each of which independently degrades management's ability to maintain effective organizational oversight.

1.5.1 Absence of Real-Time KPI Computation

Key performance indicators - Average Approval Time, Bottleneck Rate (decisions stalled beyond SLA thresholds as a percentage of total active decisions), Compliance Rate, Rejection Rate, and Decision Throughput , are computed manually through spreadsheet-based processes. This

methodology is structurally incapable of producing the refresh frequency required for real-time governance oversight. By the time a manually compiled KPI report reaches a decision-maker, the underlying operational data may already reflect a materially different state. Performance degradation that develops gradually over days goes entirely undetected between reporting cycles.

1.5.2 Absence of Multi-Metric Anomaly Detection

No automated mechanism exists to identify when a department's decision-processing behavior deviates significantly from its historical baseline across multiple simultaneous metrics. A single delayed decision may be operationally unremarkable; however, a department simultaneously exhibiting an average approval cycle time of 9.8 days against a historical baseline of 2.3 days, combined with a rejection rate rising from 5% to 29%, constitutes a statistically significant governance anomaly. Such multi-metric behavioral deviations are invisible to manual monitoring and compound silently until they escalate into compliance failures or missed organizational commitments.

1.5.3 Absence of Predictive Delay Forecasting

Management responds to workflow bottlenecks exclusively after delays have already accumulated and escalated. No mechanism exists to forecast which departments are likely to experience approval congestion in the coming weeks based on current decision intake rates and historical throughput patterns. This reactive posture means that preventive resource reallocation, escalation of pending approvals, or policy adjustments cannot be initiated in advance interventions occur only after the bottleneck has already impacted organizational output.

1.5.4 Absence of Continuous Automated Risk Scoring

Departmental risk levels in most enterprise governance frameworks are assessed manually by compliance officers on a quarterly cadence. This creates a structural monitoring gap of up to 90 days during which a department may deteriorate from a low-risk classification to a critical operational state without triggering any alert. The absence of a continuously updated, data-driven risk scoring mechanism means that high-risk states persist undetected between audit cycles, exposing the organization to compliance violations and unmanaged operational failures.

1.5.5 Absence of Automated Report Generation

Governance reports are produced through manual data extraction, reformatting in spreadsheet tools, and distribution via email. This process introduces transcription errors, consumes significant analyst effort per reporting cycle, and makes reports unavailable on demand. Decision-makers requiring an immediate governance summary must wait for the next scheduled report cycle regardless of operational urgency, delaying evidence-based management response.

1.5.6 Fragmentation of Cross Department or Module Governance Data

Decision lifecycle data, compliance violation records, and policy definitions are distributed across independent organizational systems with no unified analytical layer. Executives and department heads must consult multiple interfaces to construct a complete picture of governance performance, and even then, the view is static rather than live. The absence of a centralized observability layer prevents cross-dimensional analysis for example, correlating approval delays with compliance violations across the same department over the same time window which is precisely the class of insight required for proactive governance.

1.6 Solution Strategy

The Gov-Vision Analytics Module addresses each identified failure point through a corresponding technical approach, implemented as an independent analytical microservice that continuously ingests cross-module decision data and transforms it into real-time governance intelligence.

1.6.1 Automated Real-Time KPI Computation

Manual KPI computation is replaced by an automated aggregation engine that continuously reads raw decision and compliance data, computes all governance performance indicators, and stores the results in a time-series snapshot store. KPI data is refreshed both on a scheduled interval and immediately upon receiving state-change notifications from the decision management and compliance modules, ensuring that the dashboard always reflects the current operational state. A caching layer with a defined expiry window is applied to high-frequency dashboard queries, maintaining low latency without incurring repeated computation costs on every poll.

1.6.2 Unsupervised Anomaly Detection via Isolation Forest

Multi-metric anomaly detection is achieved by training an Isolation Forest model on historical decision records. Isolation Forest is selected because it detects anomalies by measuring how quickly a data point can be isolated from the rest of the population in a random partitioning tree anomalous decision, being statistically rare and extreme, are isolated in fewer splits and therefore receive higher anomaly scores. Six behavioral features are used: approval cycle time, rejection count, revision count, days over SLA, stage count, and submission hour. Detected anomalies are classified into severity tiers — Critical, High, Medium, and Low — based on normalized score thresholds, and surfaced to managers as real-time dismissible alerts on the executive dashboard, enabling immediate investigation rather than post-incident discovery.

1.6.3 Time-Series Delay Forecasting via Prophet

Predictive capability is introduced through a Prophet time-series forecasting model trained per department on historical decision throughput data. Prophet is selected for its ability to decompose irregular business time-series into trend and seasonality components without requiring stationary input data. The model produces 7-day, 14-day, and 30-day decision volume forecasts with confidence interval bounds, which are visualized as area charts on the management dashboard. This enables governance teams to identify departments approaching capacity thresholds weeks in advance and take preventive action — escalating approvals, reallocating reviewer bandwidth, or adjusting SLA windows — before bottlenecks affect organizational output.

1.6.4 Supervised Risk Classification via Random Forest

Continuous departmental risk scoring is achieved through a supervised Random Forest classifier trained on labeled historical governance records. Random Forest is selected for its robustness to noisy features, its native support for multi-class classification, and its built-in feature importance mechanism which provides explainability enabling compliance officers to understand not just a department's risk label but which governance factors are driving it. The model executes on a 30-minute refresh cycle, producing a live risk label (Low, Medium, High, or Critical) for each active department alongside a feature importance breakdown. This eliminates the quarterly manual audit

cycle and ensures that deteriorating departments are flagged continuously rather than only at scheduled review intervals.

1.6.5 On-Demand Report Generation

Manual report production is replaced by a programmatic report generation engine that assembles governance reports in PDF, Excel, and CSV formats on demand or on a user-defined schedule. Reports are constructed directly from pre-aggregated governance data KPI summaries, anomaly logs, compliance rates, and risk classifications eliminating manual extraction and transcription. Scheduled report delivery is configurable per report type, enabling weekly executive summaries, monthly compliance reports, and ad-hoc departmental performance snapshots to be generated and distributed automatically without analyst intervention.

1.6.6 Unified Real-Time Governance Observation Layer

Cross-module/department data fragmentation is resolved by establishing the Gov-Vision Analytics Module as the single observability layer of the platform. Decision lifecycle data and compliance records from the upstream modules are consumed through direct read access to the shared database cluster, avoiding the latency and pagination overhead of inter-service REST calls for bulk analytical queries. All derived outputs KPI snapshots, anomaly classifications, risk scores, and forecasts are consolidated and exposed through a unified dashboard interface comprising dedicated views for executive KPI monitoring, AI-powered anomaly insights, departmental risk heatmaps, delay forecasts, and compliance analytics. This provides executives and department heads with a single, continuously updated, role-gated interface through which the complete governance performance of the organization is visible at all times.

1.7 Software Requirement Specifications (SRS)

This section lists the complete requirements for the Gov-Vision Analytics Module the intelligence and observability layer of the Gov-Vision Digital Decision and Governance Platform. Requirements are divided into two categories: functional requirements, which define what the system shall do including real-time KPI computation, AI-driven anomaly detection, predictive delay forecasting, risk scoring, automated report generation, and cross-module data

integration and non-functional requirements, which define how the system shall behave in terms of security, performance, reliability, scalability, maintainability, and availability

1.7.1 Functional Requirements

R1: Executive KPI Dashboard

Description: The system shall provide a real-time executive dashboard that computes and displays ten governance Key Performance Indicators across all organizational departments, enabling management to monitor decision-making performance without manual data compilation.

Input: Decision records from m1_decisions, compliance violation records from m2_violations, a department filter parameter, and a date range supplied through the dashboard filter bar.

Output: Ten live KPI metric cards - Total Decisions, Approval Rate (%), Rejection Rate (%), Average Approval Time (hours), Bottleneck Rate (%), Compliance Rate (%), Violation Count, Decision Throughput (decisions/day), Anomaly Count, and AI Risk Score (label) updated in real time.

Process: KPI computation is event-driven. Whenever a decision state change occurs in Module 1 or a compliance violation is raised or resolved in Module 2, an authenticated webhook payload is dispatched to the respective endpoint. On receipt, the system invalidates the corresponding Redis cache keys and immediately invokes the KPI aggregation engine, which computes all ten metrics from m1_decisions and m2_violations and upsets the result into m3_kpi_snapshots. The updated KPI payload is then pushed directly to connected dashboard clients via a Socket.io kpi: updated event, triggering an immediate re-render of all KPI cards without any polling interval.

METHOD	ENDPOINT	DESCRIPTION
GET	/api/analytics/kpi-summary	Returns all ten KPI values for the requested department and date range; served from Redis cache (TTL) on hit
GET	/api/analytics/decision-volume	Returns decision counts grouped by daily, weekly, or monthly granularity
GET	/api/analytics/cycle-time-histogram	Returns decision counts across four cycle time buckets (0–24h, 24–48h, 48–72h, >72h)

GET	/api/analytics/compliance-trend	Returns per-department compliance rate time-series for multi-line chart rendering
POST	/api/events/decision-update	Receives decision state-change events from Module 1; invalidates Redis KPI cache and triggers re-aggregation
POST	/api/events/compliance-update	Receives compliance violation events from Module 2; invalidates department-scoped Redis KPI cache and triggers re-aggregation

Table 2: APIs of the KPI Dashboard Sub-Module

R2. Anomaly Detection and Alert System

Description: The system shall automatically detect abnormal patterns in both active and completed decisions using a trained Isolation Forest model, classify each flagged decision by severity, and surface unacknowledged anomalies to managers as real-time alerts while intervention is still actionable.

Input: Decision feature vectors extracted from m1_decisions cycleTimeHours, rejectionCount, revisionCount, daysOverSLA, stageCount, and hourOfDaySubmitted covering all decisions within the last 30 days regardless of completion status.

Output: Per-decision anomaly result containing anomalyScore (normalized float, 0–1), isAnomaly (boolean), and severity label (Critical, High, Medium, Low, or Normal). Anomalous results are persisted in m3_anomalies and displayed as dismissible severity-coded alerts on the executive dashboard and AI Insights page, with active decisions prioritized as they remain actionable.

Process: Detection operates through two mechanisms. The primary mechanism is event-driven on every decision state transition, the incoming feature vector is immediately evaluated against the Isolation Forest model, ensuring active decisions with abnormal daysOverSLA or cycleTimeHours are flagged within seconds. The secondary mechanism is a daily midnight cron job that re-evaluates the full 30-day decision window as a fault-tolerance reconciliation pass for any decisions missed due to dropped webhooks. In both paths, the FastAPI service normalizes input vectors using the persisted StandardScaler, scores via the Isolation Forest model, and maps results to severity thresholds (≥ 0.95 = Critical, ≥ 0.90 = High, ≥ 0.80 = Medium, ≥ 0.70 = Low).

Confirmed anomalies are upserted into m3_anomalies and pushed to connected clients via a Socket.io anomaly:new event.

METHOD	ENDPOINT	DESCRIPTION
GET	/api/ai/anomalies	Returns all unacknowledged anomaly records grouped by severity, sorted descending by anomalyScore. Active decisions are surfaced before completed ones
PUT	/api/ai/anomalies /:id/acknowledge	Sets isAcknowledged to true, records acknowledgedBy and acknowledgedAt, invalidates anomaly Redis cache
POST	/ml/anomaly/predict	FastAPI accepts decision feature array, runs Isolation Forest inference, returns normalized scores and severity labels
POST	/ml/models/train	FastAPI spawns train_isolation_forest.py as a background subprocess for weekly scheduled model retraining

Table 3: APIs of the Anomaly Detection Sub-Module

R3. Predictive Delay Forecasting

Description: The system shall forecast future decision throughput per department over 7-day, 14-day, and 30-day horizons using a trained Prophet time-series model, enabling management to identify departments approaching bottleneck conditions before delays materialize.

Input: Historical KPI snapshot records from m3_kpi_snapshots, with snapshotDate as the time index and totalDecisions as the target variable, submitted per department to the FastAPI ML microservice.

Output: Forecast arrays stored in m3_forecasts containing predicted decision count (yhat), lower confidence bound (yhat_lower), and upper confidence bound (yhat_upper) at the 80% interval for each future date. Output is visualized as a time-series area chart with confidence bands on the Forecast Dashboard using Apache ECharts.

Process: A weekly cron job queries all active departments, assembles their historical throughput time-series from m3_kpi_snapshots, and posts to the FastAPI forecast endpoint. The service fits one Prophet model per department and generates forecasts across all three horizons. Prophet requires sufficient historical data accumulation between runs to produce updated forecasts a weekly cadence aligns with this requirement since daily decision volumes do not shift multi-week trend patterns within shorter intervals. Results are inserted into m3_forecasts and a forecast:updated Socket.io event is emitted to connected clients.

METHOD	ENDPOINT	DESCRIPTION
GET	/api/analytics/forecasts/:deptId	Returns stored forecast data for the specified department across all three horizons
POST	/ml/forecast/predict	FastAPI accepts department time-series, fits Prophet model, returns forecast arrays for all configured horizons

Table 4: APIs of the Predictive Forecasting Sub-Module

R4: AI-Driven Risk Scoring

Description: The system shall assign a continuously updated risk classification to each active department by applying a trained Random Forest classifier to live governance feature vectors, providing management with an explainable departmental risk signal.

Input: Per-department governance feature vector assembled from m3_kpi_snapshots and m2_violations approval delay ratio, SLA breach frequency, violation count per decision, policy compliance rate, and revision cycle count.

Output: A risk label (Low, Medium, High, or Critical) and a feature importance breakdown per department indicating the percentage contribution of each governance feature to the composite risk score. Results are written to m3_kpi_snapshots.riskScore and riskLevel, and visualized as a department-by-category ECharts heatmap with drill-down capability.

Process: A daily cron job queries the latest governance feature vector for each active department from m3_kpi_snapshots and m2_violations and posts to the FastAPI risk scoring endpoint. The service runs RandomForestClassifier inference and returns a risk label alongside feature importance scores for each department. Results are written back to m3_kpi_snapshots and a risk:updated Socket.io event is emitted to connected clients. Risk scoring runs daily since the

underlying features approval delay ratios, violation counts, compliance rates are aggregated metrics that accumulate over days and do not shift within shorter intervals.

METHOD	ENDPOINT	DESCRIPTION
GET	/api/ai/risk-scores	Returns current risk label and feature importance breakdown for all active departments
POST	/ml/risk/score	FastAPI accepts department feature vectors, runs RandomForestClassifier inference, returns risk labels and feature importance arrays

Table 4: APIs of the Risk Scoring Sub-Module

R5. Automated Report Generation

Description: The system shall enable on-demand and scheduled generation of structured governance reports in PDF, Excel, and CSV formats, assembling data programmatically from pre-aggregated governance collections without manual extraction.

Input: Report parameters report name, type (Executive, Compliance, Decision, Department, or Custom), output format (PDF, Excel, or CSV), date range, and target departments. For scheduled reports, a user-defined cron expression and recipient email list are additionally configured through m3_report_schedules.

Output: A generated report file persisted to the server filesystem with its path recorded in m3_reports. Report status transitions through Generating → Completed or Failed. Completed reports are available for immediate download through the Report History page.

Process: On-demand reports are triggered explicitly by the user through the Report Builder interface. Scheduled reports are registered at server startup by reading all active m3_report_schedules documents and dynamically registering each cron expression with node-cron. In both cases, the report generation service queries m3_kpi_snapshots, m3_anomalies, and m2_violations for the requested parameters and assembles the output using jsPDF for PDF, ExcelJS for Excel, and json2csv for CSV. Report generation is a user-initiated or user-scheduled operation and does not respond to upstream system events.

Method	Endpoint	Description
POST	/api/reports/generate	Initiates on-demand report generation; returns reportId and initial status
GET	/api/reports/history	Returns list of all generated reports from m3_reports with download URLs
GET	/api/reports/download/:reportId	Streams the generated report file to the client
POST	/api/reports/schedules	Creates a new scheduled report entry in m3_report_schedules
GET	/api/reports/schedules	Returns all active report schedules

Table 6: APIs of the Report Generation Sub-Module

1.7.2 Non-Functional Requirements

Security: All dashboard-facing API endpoints are protected by JWT authentication enforced through validateJWT.ts, which verifies the Bearer token against the shared JWT_SECRET and attaches the decoded user payload to the request. Role-based access control through requireRole.ts restricts anomaly acknowledgement, risk score retrieval, and report generation endpoints to users with the manager or admin role. All machine-to-machine webhook endpoints are protected by serviceKey.ts, which validates the x-service-key request header against process.env.SERVICE_KEY. No credentials or service keys are transmitted in URL query parameters.

Performance: KPI and anomaly cache keys are invalidated on every upstream webhook event and immediately rewritten with fresh results, so the Redis TTL of 3600 seconds on both functions purely as a fallback for missed webhook scenarios. MongoDB aggregation pipelines include $match stages on indexed fields (department, createdAt, completedAt) to prevent full collection scans. WebSocket push delivery eliminates polling overhead on both client and server.

Reliability: All asynchronous Express route handlers incorporate try/catch blocks returning structured HTTP 500 responses rather than crashing the server process. The FastAPI startup lifecycle validates that all serialized model files are present and loadable before the service accepts inference requests. Failed report generation transitions the corresponding m3_reports document to a Failed status with an error description. All scheduled cron jobs log execution start, completion, and errors on every run.

Scalability: The three-tier microservices architecture allows the Node.js backend and the Python FastAPI ML microservice to be scaled independently. The Redis caching layer absorbs concurrent read traffic without proportionally increasing MongoDB query load. The ML microservice is stateless between requests, so horizontal scaling requires no session affinity.

Maintainability: TypeScript strict mode is enforced across both the backend and frontend tiers. All shared data contracts are defined in a single types/index.ts file per tier, preventing interface drift across the API boundary. All cross-module read operations are isolated to dedicated read-only Mongoose models in backend/models/external/, ensuring upstream schema changes require updates in a single clearly identified location.

Availability: If the FastAPI ML microservice is unavailable, the Node.js backend continues to serve cached KPI data and the last computed anomaly and risk records from Redis and MongoDB. If Redis is unavailable, the system falls back to direct MongoDB queries. No single downstream dependency blocks core dashboard functionality.

2 PROJECT PLANING

2.1 Hardware and Software Requirements

Hardware Requirements (System Specification)	
Memory	16 GB RAM
Processor	AMD Ryzen 7 7840HS
Storage	512 GB SSD
Graphics Card	NVIDIA GeForce RTX 3050

Software Requirements	
Operating System	Windows 11
Frontend	React.js, HTML5, CSS3, TailwindCSS
Backend	Node.js, Express.js, FastAPI
AI / ML	Python, scikit-learn, Prophet, Pandas, NumPy
Analytics & Visualization	Recharts, Apache ECharts
Database	MongoDB (Mongoose), Redis
Report Generation	jsPDF, ExcelJS, json2csv
Scheduling	node-cron
Development Tools:	VS Code, npm, pip

API Testing:	Thunder Client
Version Control	Git, GitHub

2.2 Team Structure

Fig 1: Team Structure

2.3 Software Development Life Cycle (SDLC)

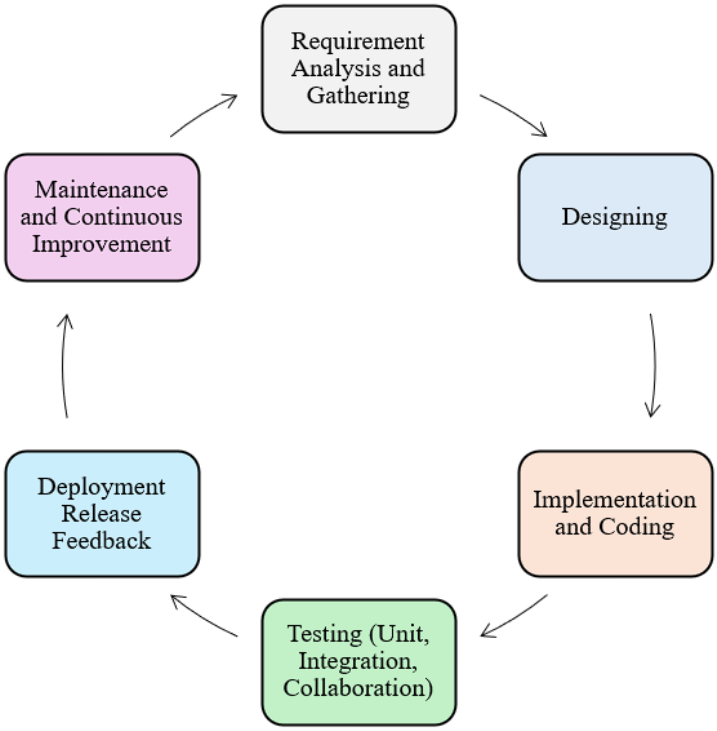

Fig 2: Agile Software Development Life Cycle

Description: Analytics, Reporting & AI/ML Monitoring System is developed using the Agile SDLC, where features are built iteratively. Each cycle focuses on key functionalities such as real-time KPI dashboards, anomaly detection, forecasting, risk scoring, and report generation. Backend services handle KPI aggregation, AI/ML

models (Isolation Forest, Random Forest, Prophet), and REST APIs, while frontend components like the Executive Dashboard, AI Insights, and Report Builder are developed in parallel. After testing, features are deployed and continuously improved using feedback from integration with other Modules.

2.4 Gantt–chart

Fig 3: Gantt Chart

3 DESIGN STRATEGY FOR THE SOLUTION

3.1 Architecture Diagram

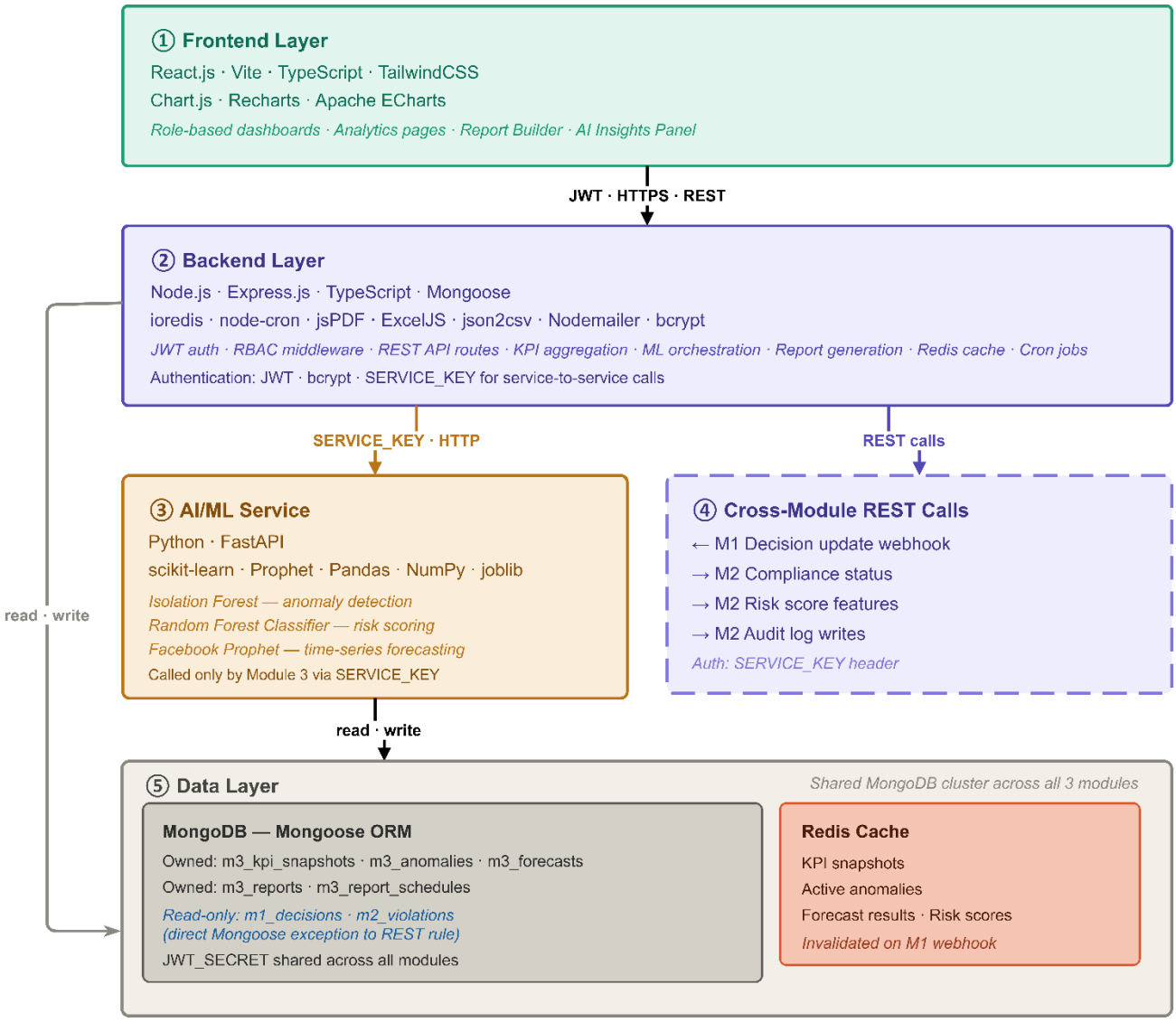

Fig 4: Architecture Overview Diagram of Analytics, Reporting & AI/ML Monitoring System

Description: The layered architecture of the Analytics, Reporting & AI/ML Monitoring System of the GovVision platform. It shows how the frontend (React + dashboards) communicates with the Node.js backend via REST APIs secured by JWT, which handles KPI computation, report generation, and orchestration. The backend interacts with a Python Fast API AI/ML service (using Isolation Forest, Random Forest, and Prophet) for anomaly detection, risk scoring, and forecasting. It also integrates with other modules through secure REST calls. Data is stored in a shared MongoDB cluster, while Redis caching improves performance for KPIs, anomalies, and forecasts.

4 PROGRESS TILL DATE

4.1 Analytics Dashboard

The executive analytics dashboard is implemented and operational. [Fig 5] It renders ten live KPI metric cards: Total Decisions (2,500), Approval Rate (76.3%), Rejection Rate (18.9%), Average Approval Time (48h), Bottleneck Rate (2.6%), Compliance Rate (66.1%), Violation Count (0), Decision Throughput (3/day), Anomaly Count (5), and AI Risk Score (Low). The top bar provides a date range picker, a department selector, and an Export Data CSV snapshot action. The sidebar exposes navigation routes for Dashboard, Deep Insights, Anomaly Detection, Forecast, Risk Assessment, and Reports.

[Fig 6] Three analytical charts are rendered below the KPI cards: a Decision Throughput and Volume Recharts line/bar chart with Daily/Weekly/Monthly granularity toggle, an Avg Approval Time Distribution histogram bucketing decisions across 0–24h, 24–48h, 48–72h, and >72h, and a Compliance Rate Trend Apache ECharts multi-line chart plotting monthly compliance rate per department (CS005, FI001, HR002, IT004, OP003) against a 95% target line. A Real-Time Anomalies panel displays 5 active Isolation Forest anomaly alerts across Human Resources, Finance, Customer Service, and Operations, each with an anomalyScore, severity label, and Acknowledge action.

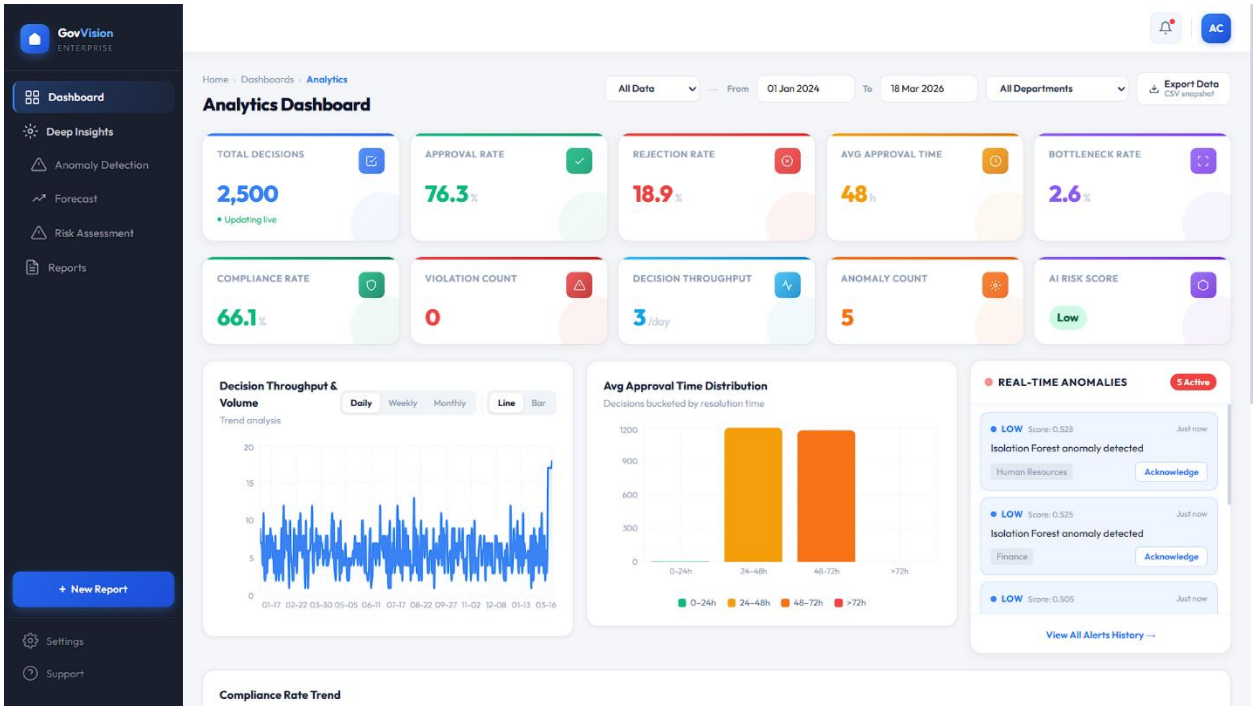

Fig 5: Analytics Dashboard — KPI cards, chart panels, and real-time anomaly feed

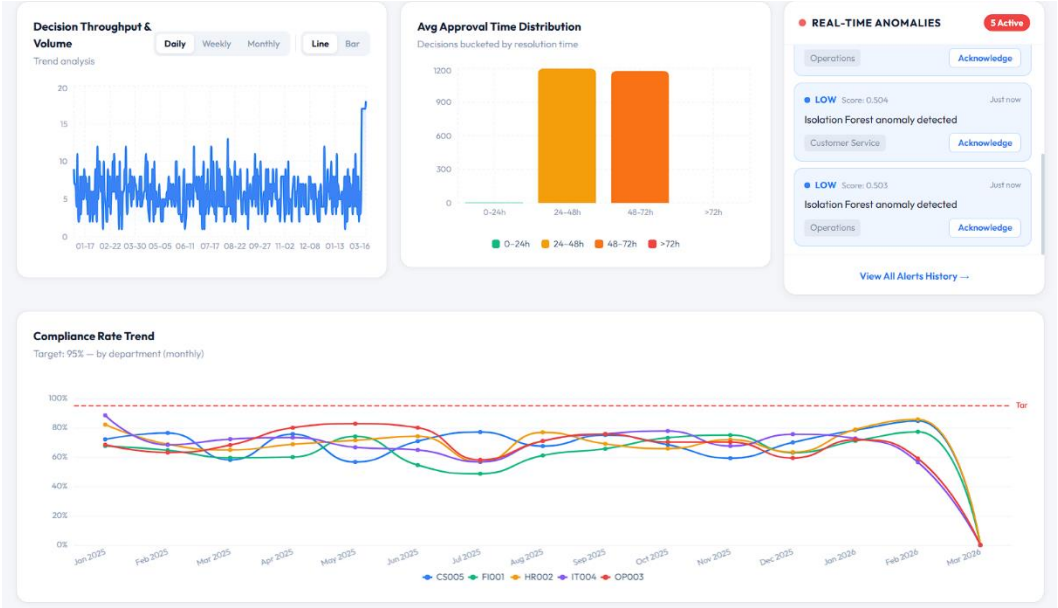

Fig 6 Analytics Dashboard - Compliance Rate Trend,Decision Throughput, Avg Approval Time Distribution and anomaly feed continued

4.2 Data Ingestion -CSV Import Pipeline

Decision data was ingested from the dataset AI_Workflow_Optimization_Dataset_2500_Rows_v1.csv into the m1_decisions MongoDB collection using a custom import pipeline implemented in importCSV.ts. The pipeline processes the source CSV through five stages: field mapping, derived field computation, department normalization, timestamp remapping, and batched insertion

Source-to-Schema Field Mapping

The table below documents every source CSV column, the target m1_decisions schema field it was mapped or converted to, and the transformation applied.

CSV SOURCE COLUMN	TARGET SCHEMA FIELD	TYPE	TRANSFORMATION / DESCRIPTION
Decision ID	---	---	Used as import reference only; MongoDB generates _id
Task Type	status	String	Enumeration mapping: completed → approved, rejected → rejected, in-progress → pending

Created Date	createdAt	Date	Parsed to ISO timestamp; shifted into 2025–2026 window
Completed Date	completedAt	Date	Parsed to ISO timestamp; set to null for pending decisions
---	cycleTimeHours	Number	Computed: (completedAt − createdAt) in hours; null for pending
Rejection Count	rejectionCount	Number	Mapped directly from source column
Revision Count	revisionCount	Number	Mapped directly from source column
---	daysOverSLA	Number	Derived: decision duration minus defined SLA threshold in days; computed live for pending
Stage Count	stageCount	Number	Mapped directly from source column
Department	departmentId	String	Normalized to canonical identifier (see department mapping table below)
Department	departmentName	String	Normalized to canonical human-readable name
Submission Hour	hourOfDaySubmitted	Number	Mapped directly; integer 0–23
Priority	priority	String	Mapped directly: low, medium, or high

Table 7: CSV Source Column to m1_decisions Schema Field Mapping

Department Normalization

Source department strings contained variations and aliases across rows. All references were normalized to five canonical string identifiers during import.

SOURCE VALUE(S)	CANONICAL ID	DEPARTMENT NAME
Finance, FI, finance	FI001	Finance
HR, Human Resources, hr	HR002	Human Resources
Operations, Ops, operations	OP003	Operations
IT, Information Technology, it	IT004	Information Technology
CS, Customer Service, customer service	CS005	Customer Service

Table 8: Department Normalization Mapping

Timestamp Remapping

All source timestamps were shifted into the 2025–2026 window to ensure all records fall within the default dashboard date filter range. The relative time difference between createdAt and completedAt was preserved during the shift to maintain accurate cycleTimeHours values.

4.3 Database Model - MongoDB Collections

Three MongoDB collections have been created and are operational. One cross-module read-only Mongoose models reference the upstream decision and compliance collections.

4.3.1 m1_decisions (Cross-Module Read-Only)

The m1_decisions collection is the primary data source for all KPI aggregation and anomaly detection. It is populated with 2,500 decision records imported via a CSV ETL pipeline from the dataset AI_Workflow_Optimization_Dataset_2500_Rows_v1.csv. The import pipeline parsed each row, mapped source columns to the target schema, normalized department references to five canonical identifiers (FI001, HR002, OP003, IT004, CS005), computed derived fields (cycleTimeHours from timestamp difference, daysOverSLA from SLA threshold deviation), and remapped all timestamps into the 2025–2026 window.

FIELD	TYPE	DESCRIPTION
_id	ObjectId	Auto-generated primary key
status	String	Decision state: approved, rejected, or pending
createdAt	Date	Submission timestamp
completedAt	Date	Terminal state timestamp; null for pending decisions
cycleTimeHours	Number	Computed: (completedAt − createdAt) in hours; null for pending
rejectionCount	Number	Number of rejections accumulated across review stages
revisionCount	Number	Number of revision cycles before terminal state
daysOverSLA	Number	Days beyond the SLA threshold; computed live for pending

stageCount	Number	Total approval stages traversed
departmentId	String	Canonical department identifier
departmentName	String	Human-readable department name
hourOfDaySubmitted	Number	Hour of submission (0–23)
priority	String	low, medium, or high

Table 9: m1_decisions - Field Definitions


```
_id: ObjectId('69b9a452a784954b19dabeab')
status : "approved"
createdAt : 2025-01-29T23:36:20.794+00:00
completedAt : 2025-01-31T22:12:20.794+00:00
cycleTimeHours : 46.6
rejectionCount : 9.3
revisionCount : 3
daysOverSLA : 0
stageCount : 3
departmentId : "CS005"
departmentName : "Customer Service"
hourOfDaySubmitted : 5
priority : "low"
```


Fig. 7: MongoDB - m1_decisions

4.3.2 m3_kpi_snapshots

Stores computed KPI aggregations per department per snapshot date.

FIELD	TYPE	DESCRIPTION
_id	ObjectId	Auto-generated primary key
snapshotDate	Date	Date of aggregation
departmentId	String	Department identifier or "ORG" for organisation-level
totalDecisions	Number	Total decisions in the aggregation window
approvedCount	Number	Decisions with status approved

rejectedCount	Number	Decisions with status rejected
pendingCount	Number	Decisions with status pending
avgCycleTimeHours	Number	Mean cycle time of completed decisions in hours
complianceRate	Number	Percentage of decisions satisfying SLA and policy constraints
anomalyCount	Number	Count of unacknowledged anomalies for this department
bottleneckCount	Number	Pending decisions with daysOverSLA exceeding the threshold
bottleneckRate	Number	bottleneckCount as a percentage of total active decisions
violationCount	Number	Total compliance violations recorded
openViolations	Number	Unresolved violations
riskLevel	String	AI-assigned label: low, medium, high, or critical
riskScore	Number	Numeric risk score from Random Forest classifier

Table 10: m3_kpi_snapshots -Field Definitions


```
_id: ObjectId('69b98b7f922443f67a4fd345')
snapshotDate : 2026-03-16T18:30:00.000+00:00
departmentId : "ORG"
__v : 0
anomalyCount : 16
approvedCount : 493
avgCycleTimeHours : 3.014399999999996
complianceRate : 5
openViolations : 0
pendingCount : 1503
rejectedCount : 504
riskLevel : "low"
riskScore : 0
totalDecisions : 2500
violationCount : 0
bottleneckCount : 263
bottleneckRate : 10.5
▸ bottleneckThresholds : Object
```


Fig. 8: MongoDB - m3_kpi_snapshots organisation-level snapshot document

Description: The organisation-level snapshot dated 2026-03-16 contains: totalDecisions: 2,500, approvedCount: 493, rejectedCount: 504, pendingCount: 1,503, avgCycleTimeHours:

3.01, complianceRate: 5, anomalyCount: 16, bottleneckCount: 263, bottleneckRate: 10.5, violationCount: 0, riskLevel: "low", riskScore: 0.

4.3.3 m3_anomalies

Stores one document per flagged decision, upserted on each anomaly detection run..

FIELD	TYPE	DESCRIPTION
_id	ObjectId	Auto-generated primary key
decisionId	ObjectId	FK reference to m1_decisions._id
anomalyScore	Number	Normalized anomaly score (0–1)
department	String	Department of the flagged decision
severity	String	Critical, High, Medium, Low, or Normal
isAcknowledged	Boolean	False until a manager acknowledges the alert
acknowledgedBy	ObjectId	FK reference to users._id; null until acknowledged
acknowledgedAt	Date	Acknowledgement timestamp; null until acknowledged
description	String	Human-readable description of the flagged pattern
featureValues	Object	Raw six-feature values used during inference

Table 11: m3_anomalies - Field Definitions

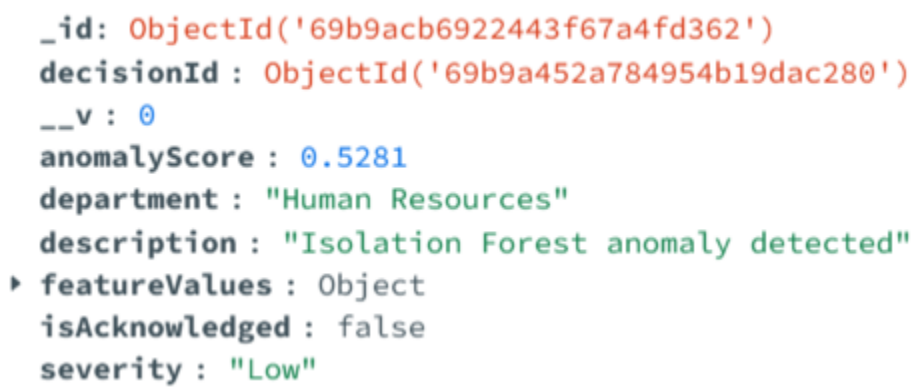

Fig. 6: MongoDB - m3_anomalies document
(Human Resources, anomalyScore: 0.5281, severity: Low)

Description: Fig 6 refers m1_decisions, anomalyScore: 0.5281, department: "Human Resources", severity: "Low", isAcknowledged: false.

4.4 API Testing - Thunder Client

All implemented REST API endpoints have been verified using Thunder Client within VS Code.

METHOD	ENDPOINT	AUTH	TEST INPUT	RESPONSE	STATUS
GET	/health	None	—	200 OK	PASS
GET	/health (FastAPI :8000)	None	—	200 OK, model loaded	PASS
GET	/api/analytics/kpi-summary	JWT	All Departments	200 OK, ten KPI values	PASS
GET	/api/analytics/decision-volume	JWT	granularity=daily	200 OK, count array	PASS
GET	/api/analytics/cycle-time-histogram	JWT	---	200 OK, four bucket counts	PASS
GET	/api/analytics/compliance-trend	JWT	---	200 OK, per-dept time-series	PASS
GET	/api/ai/anomalies	JWT	---	200 OK, records by severity	PASS
POST	/api/events/decision-update	x-service-key	{ newStatus: "approved", department: "CS005" }	200 OK, cache invalidated	PASS
POST	/api/events/decision-update	Missing key	---	401 Unauthorized	PASS
POST	/ml/anomaly/predict	x-service-key	cycleTimeHours: 3, rejectionCount: 2, revisionCount: 5, daysOverSLA: 0, stageCount: 2, hourOfDaySubmitted: 11	200 OK - score: 0.4085, isAnomaly: false, severity: Normal	PASS
POST	/ml/anomaly/predict	x-service-key	cycleTimeHours: 148, rejectionCount: 12,	200 OK - score: 1.000, isAnomaly:	PASS

			revisionCount: 10, daysOverSLA: 5, stageCount: 8, hourOfDaySubmitted: 3	true, severity: Critical	
POST	/ml/anomaly/predict	Invalid key	—	401 Unauthorized	PASS

Table 12: Thunder Client API Test Results

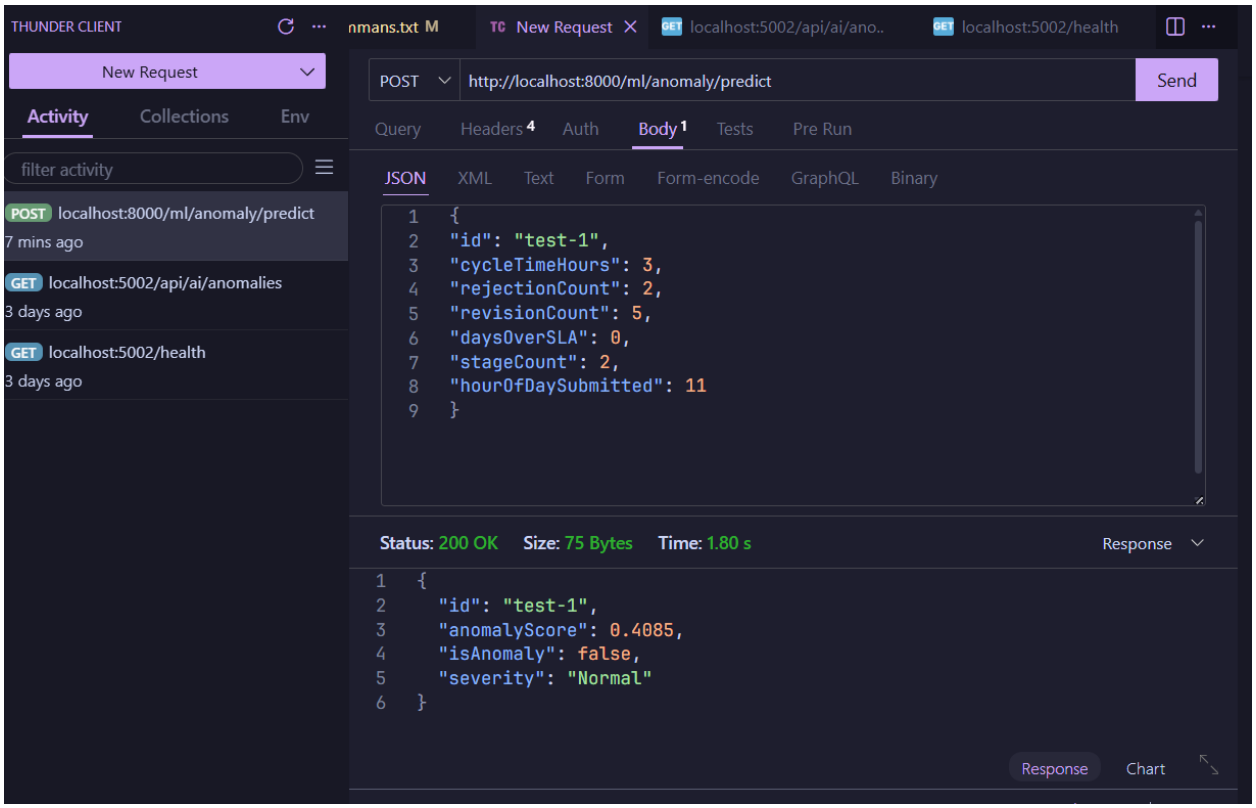

Fig. 7: Thunder Client - POST /ml/anomaly/predict with normal input values returning anomalyScore: 0.4085, isAnomaly: false, severity: Normal

4.5 Isolation Forest Model Training

The Isolation Forest training pipeline was executed on the full 2,500 record m1_decisions dataset. The training process completed in six documented steps. [All results depicted in Fig 8]

Step 1 - Data Load: 2,500 documents loaded from MongoDB. Feature matrix shape confirmed as (2,500, 6).

Step 2 - Feature Matrix Construction: Six features extracted per decision: cycleTimeHours, rejectionCount, revisionCount, daysOverSLA, stageCount, and hourOfDaySubmitted. 2,500 feature rows used for training.

Step 3 -Normalization: StandardScaler applied via fit_transform(). Training mean: [47.92, 4.36, 6.29, 0.12, 2.13, 11.72]. Training standard deviation: [13.59, 2.33, 3.35, 0.45, 0.82, 6.9]. Scaler fit: PASS.

Step 4 Model Training: IsolationForest trained with contamination: 0.06. Training complete: PASS. 150 decisions flagged on training set (6.0%). Model artifacts saved to ml_service/models/isolation_forest.pkl and ml_service/models/isolation_forest_scaler.pkl.

Step 5 Unsupervised Score Inference: All 2,500 decisions scored. Score mean: 0.2491. Score standard deviation: 0.1534. Score range: 0.0000 to 1.0000.

Step 6 Validation Visualizations: Five diagnostic charts generated to verify model behavior and score distribution.

The severity band distribution across all 2,500 decisions is as follows:

SCORE BAND	SEVERITY LABEL	DECISION COUNT	PERCENTAGE	ANOMALIES FLAGGED
0.00 – 0.20	Very Normal	1,103	44.1%	0
0.20 – 0.50	Normal	1,218	48.7%	0
0.50 – 0.70	Borderline	142	5.7%	113
0.70 – 0.80	Low	16	0.6%	16
0.80 – 0.90	Medium	12	0.5%	12
0.90 – 0.95	High	6	0.2%	6
0.95 – 1.00	Critical	3	0.1%	3
Total		**2,500**	**100%**	**150 .0%)**

Table 13: Isolation Forest — Severity Band Distribution

Results from the Isolation Forest Model Training

```
1. Load Data From MongoDB
------------------------------------------------------------------------
Documents loaded              : 2500
Feature matrix shape          : (2500, 6)

2. Build Feature Matrix
------------------------------------------------------------------------
Feature rows used for training: 2500

3. Normalize Data
------------------------------------------------------------------------
Scaler fit                    : PASS
Training mean                 : [47.92  4.36  6.29  0.12  2.13 11.72]
Training standard deviation   : [13.59  2.33  3.35  0.45  0.82  6.9 ]

4. Train Isolation Forest on Full Dataset
------------------------------------------------------------------------
Contamination setting         : 0.06
Training complete             : PASS
Flagged on training set       : 150 (6.0%)
Models saved                  : PASS

5. Unsupervised Score Inference
------------------------------------------------------------------------
Decisions scored              : 2500
Flagged anomalies             : 150 (6.0%)

6. Unsupervised Validation Visualizations
------------------------------------------------------------------------

Severity Band Summary (Normal to Critical)
------------------------------------------------------------------------
Total anomalies flagged       : 150
  Very Normal 0.00-0.20           | total: 1103 ( 44.1%)  anomalies:    0
  Normal 0.20-0.50                | total: 1218 ( 48.7%)  anomalies:    0
  Borderline 0.50-0.70            | total:  142 (  5.7%)  anomalies:  113
  Low 0.70-0.80                   | total:   16 (  0.6%)  anomalies:   16
  Medium 0.80-0.90                | total:   12 (  0.5%)  anomalies:   12
  High 0.90-0.95                  | total:    6 (  0.2%)  anomalies:    6
  Critical 0.95-1.00              | total:    3 (  0.1%)  anomalies:    3

Summary
------------------------------------------------------------------------

  Borderline 0.50-0.70            | total:  142 (  5.7%)  anomalies:  113
  Low 0.70-0.80                   | total:   16 (  0.6%)  anomalies:   16
  Medium 0.80-0.90                | total:   12 (  0.5%)  anomalies:   12
  High 0.90-0.95                  | total:    6 (  0.2%)  anomalies:    6
  Critical 0.95-1.00              | total:    3 (  0.1%)  anomalies:    3
```

```
Summary
------------------------------------------------------------

Model Run
------------------------------------------------------------
Training approach            : Full-dataset unsupervised fit
Training set                 : 2500 decisions
Full scored set              : 2500 decisions

Unsupervised outputs
------------------------------------------------------------
Flagged anomalies            : 150 (6.0%)
Score mean                   : 0.2491
Score standard deviation     : 0.1534
Score range                  : 0.0000 to 1.0000
```

Fig 8: Results from the Isolation Forest Model Training

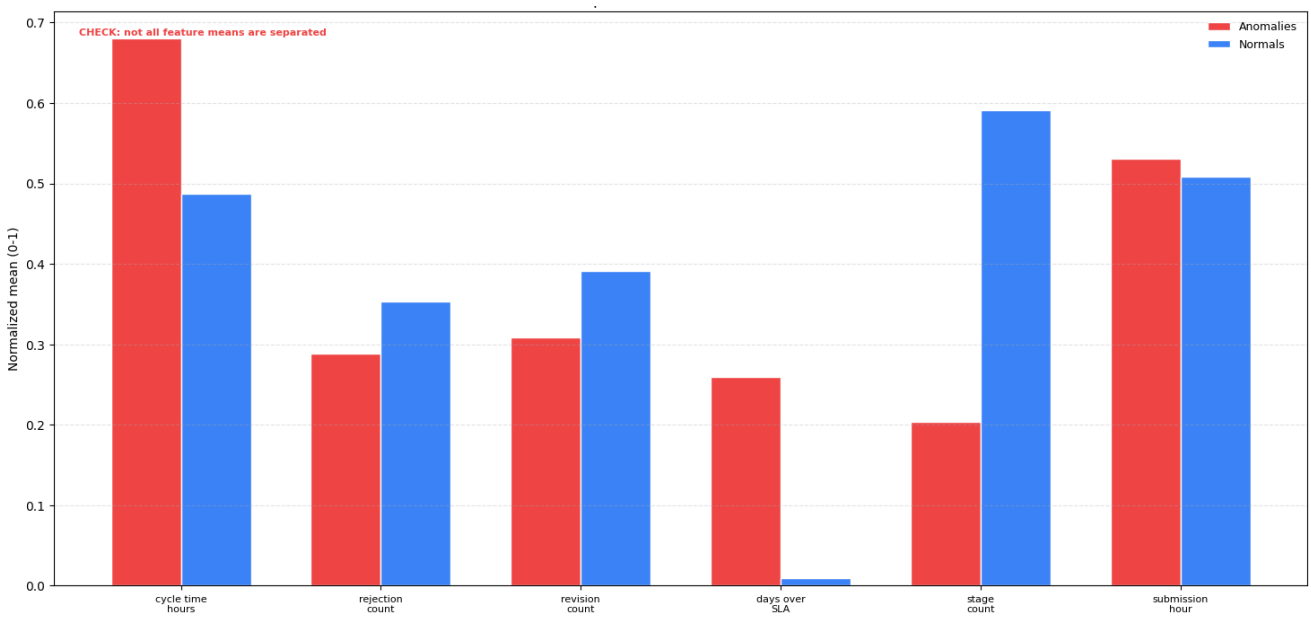

Fig. 9: Feature Separation (Predicted Anomalies vs Normals).

Description: Anomalies show higher normalized mean cycleTimeHours (0.68) against normals (0.49). Anomalies show lower stageCount (0.21) and lower daysOverSLA (0.26) compared to normals (0.59 and 0.01 respectively)

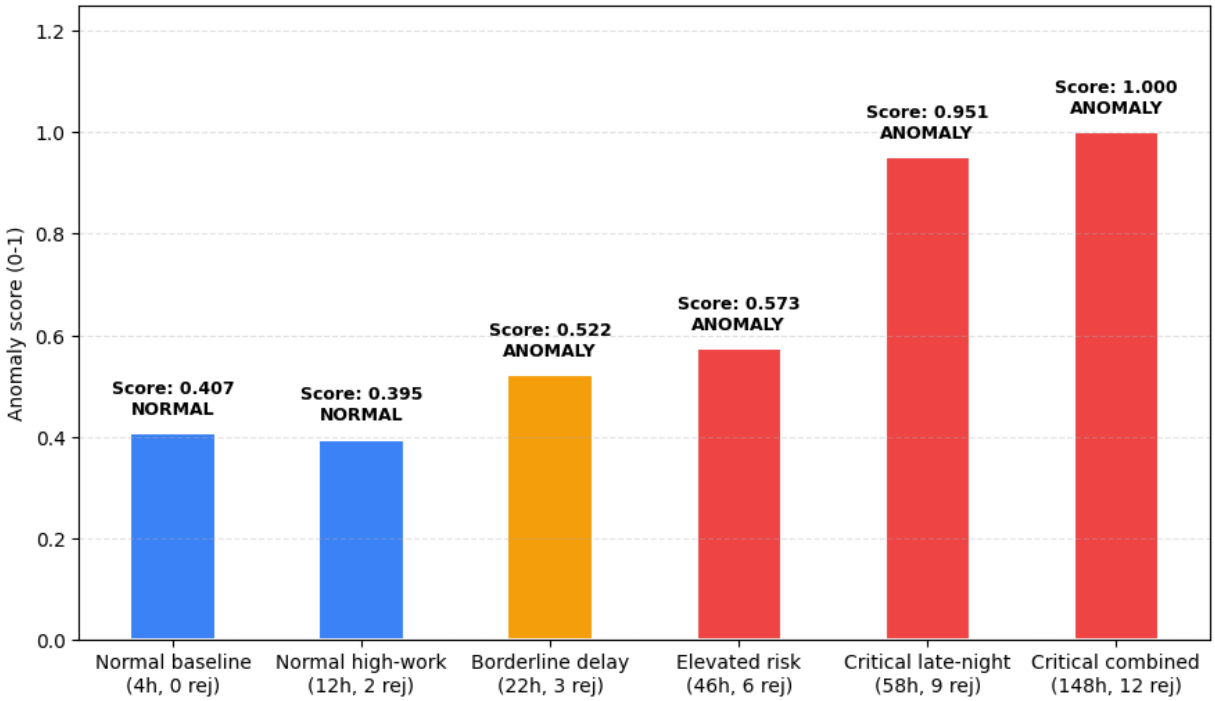

Fig. 10: Sanity Test Cases.

Description: Six synthetic inputs scored: Normal baseline (4h, 0 rejection): 0.407 NORMAL. Normal high-work (12h, 2 rejection): 0.395 NORMAL. Borderline delay (22h, 3 rejection): 0.522 ANOMALY. Elevated risk (46h, 6 rejection): 0.573 ANOMALY. Critical late-night (58h, 9 rejection): 0.951 ANOMALY. Critical combined (148h, 12 rejection): 1.000 ANOMALY

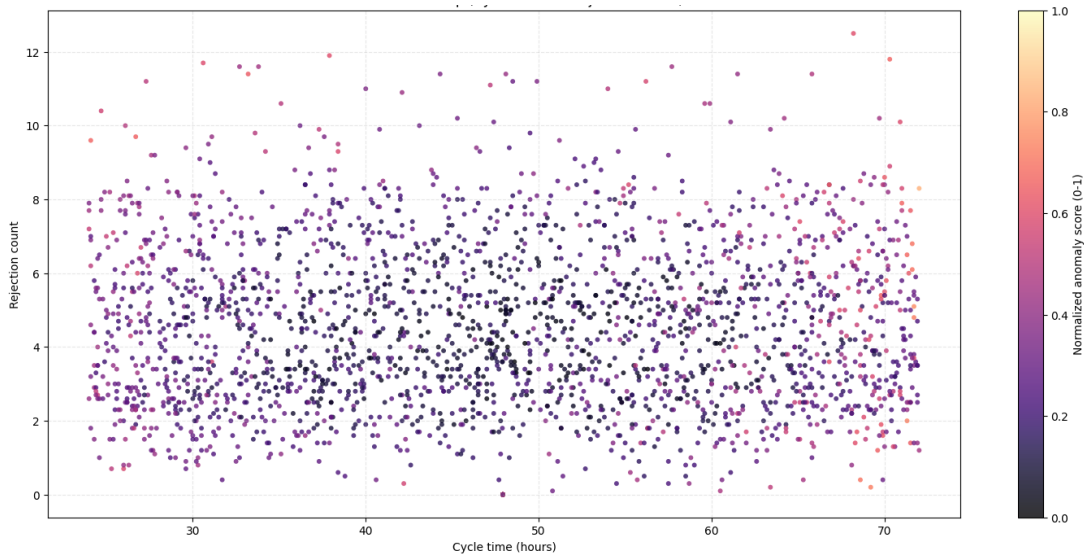

Fig. 11: Scatter Map (Cycle Time vs Rejection Count)

Description: colour-mapped by normalized anomaly score (dark purple = 0.0, yellow = 1.0). High-scoring decisions appear at elevated rejection counts (8–12) across the full cycle time range confirming multi-feature anomaly detection

5 SUMMARY AND CONCLUSION

5.1 Progress Summary

The following table summarises the implementation status of all planned components of the GovVision Analytics Module as of the current submission date.

COMPONENT	DESCRIPTION	STATUS
Project architecture	Three-tier setup: React client, Node.js backend, Python FastAPI ML service	Complete
Core middleware	validateJWT.ts, requireRole.ts, serviceKey.ts	Complete
MongoDB connection	db.ts — MongoDB Atlas connection utility	Complete
m1_decisions schema	Cross-module read-only Mongoose model	Complete
m3_kpi_snapshots schema	KPI aggregation snapshot collection	Complete
m3_anomalies schema	Anomaly detection results collection	Complete
CSV ETL pipeline	importCSV.ts — 2,500 records imported with department normalization and derived field computation	Complete
KPI aggregation engine	kpiAggregator.ts — ten KPI metrics computed via MongoDB aggregation pipelines	Complete
Webhook event routes	POST /api/events/decision-update and POST /api/events/compliance-update	Complete
Analytics API endpoints	GET /api/analytics/kpi-summary, decision-volume, cycle-time-histogram, compliance-trend	Complete
Executive Dashboard UI	Ten KPI cards, three chart components, anomaly feed panel, department and date filters	Complete
Isolation Forest training	train_isolation_forest.py — 2,500 records, six features, StandardScaler, model artifacts saved	Complete
FastAPI ML service	POST /ml/anomaly/predict and POST /ml/models/train — x-service-key protected	Complete
Anomaly API routes	GET /api/ai/anomalies and PUT /api/ai/anomalies/:id/acknowledge	Partial

Full AI route wiring	Node.js aiRoutes.ts wiring to FastAPI ML service for on-demand inference	In Progress
Authentication guards	JWT middleware implemented; full enforcement on analytics endpoints pending	Pending
m3_forecasts schema	Prophet forecast collection	Not Started
m3_reports schema	Report metadata collection	Not Started
m3_report_schedules schema	Scheduled report configuration collection	Not Started
Random Forest risk scoring	train_random_forest.py, riskJob.ts, riskScore population into m3_kpi_snapshots	Not Started
Prophet delay forecasting	train_prophet.py, forecastJob.ts, m3_forecasts population	Not Started
Report generation service	reportGenerator.ts, jsPDF, ExcelJS, json2csv	Not Started
Report Builder.	Frontend report generation and history pages	Not Started
Risk Heatmap page	RiskHeatmap.tsx, ECharts heatmap grid with department	Not Started

Table 14: GovVision Analytics Module - Progress Summary

5.2 Conclusion

The Gov-Vision Analytics Module has progressed through the foundational and core analytical layers of its planned implementation. The three-tier microservices architecture React frontend, Node.js backend, and Python FastAPI ML microservice is fully operational. The backend exposes a secured analytics API with service-key authentication on machine-to-machine endpoints, event-driven KPI aggregation triggered via webhooks from Module 1 and Module 2, and analytics routes delivering computed governance metrics directly from MongoDB aggregation pipelines.

The executive dashboard renders ten live KPI metrics computed from 2,500 imported decision records, supported by three analytical chart components and a real-time anomaly feed. The Isolation Forest anomaly detection model has been trained on the full dataset across six behavioural features, with both the trained model and Standard Scaler serialized and loaded into the FastAPI service. The model correctly classifies synthetic test cases across the full severity range from Normal (score: 0.407) to Critical (score: 1.000), and 150 decisions (6.0%) have been flagged and stored in m3_anomalies. The current 2,500-record training corpus establishes a functional anomaly boundary; however, expanding the dataset to 10,000 or more records incorporating a wider diversity of departments, rejection patterns, and submission behaviours would allow the model to construct a denser inlier distribution and improve score separation across severity bands, reducing borderline misclassification.

The remaining work full AI/ML route integration, complete JWT enforcement across all endpoints, Random Forest risk scoring, Prophet delay forecasting, automated report generation, and the Risk Heatmap visualization is planned and targeted for completion before the submission deadline. The core data pipeline, model training, and API infrastructure established in the current phase provide a stable foundation for the remaining sprint deliverables.

REFERENCES

[1] D. Yaganti, "A modular Python-based framework for real-time enterprise KPI visualization using Pandas and interactive dashboards.," *International Journal of Science and Research (IJSR),* vol. 9, no. 3, p. 1735–1736, 2020. https://dx.doi.org/10.21275/SR20033094751

[2] V. Dachepalli, "AI-driven decision support systems in ERP," *International Journal of Computer Science and Data Engineering,* vol. 2, no. 2, p. 7, 2025. http://dx.doi.org/10.55124/csdb.v2i2.248

[3] A. Herreros-Martínez, R. Magdalena-Benedicto, J. Vila-Francés, A. J. Serrano-López, S. Pérez-Díaz, and J. J. Martínez-Herráiz, "Applied machine learning to anomaly detection in enterprise purchase processes: A hybrid approach using clustering and isolation forest," *Information,* vol. 16, no. 3, p. 177, 2025. https://doi.org/10.3390/info16030177

[4] B. Huang, J. Wei, Y. Tang, and C. Liu,, "Enterprise risk assessment based on machine learning," *Computational Intelligence and Neuroscience,* no. 1, p. 6049195, 2021. https://doi.org/10.1155/2021/6049195

[5] A. Kolková and A. Ključnikov, "Demand forecasting: AI-based, statistical and hybrid models vs practice-based models – The case of SMEs and large enterprises," *Economics and Sociology,* vol. 15, no. 4, pp. 39-42, 2022. https://doi.org/10.14254/2071-789X.2022/15-4/2